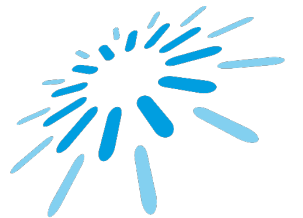




Collaboration, Quality, Test

Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences

**Recap:
Difference between regression test and confirmation
test?**

Collaboration Tools: Software Dependency Management

Quick Git Recap

Dependency Management and Build Tools

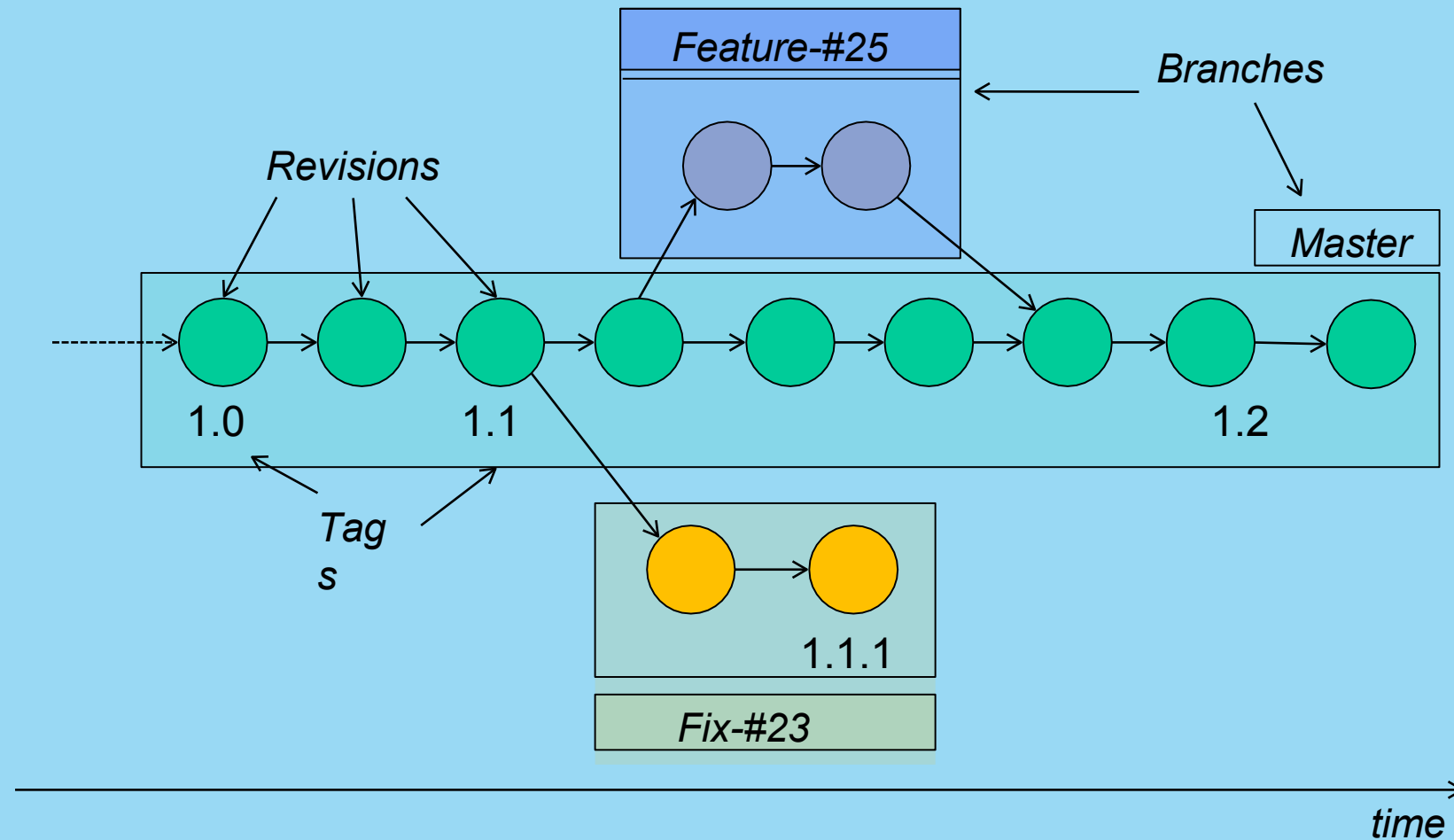
Code Demo

Source Code Management Tools

- Keep source code and branches
- Track source code changes
- Merge changes to the original code
- Basic version control features:
 - Version/release tagging
 - Tracking of change history
 - Independent, parallel development
 - Merging/conflict management
 - Storage efficiency (stores only deltas)

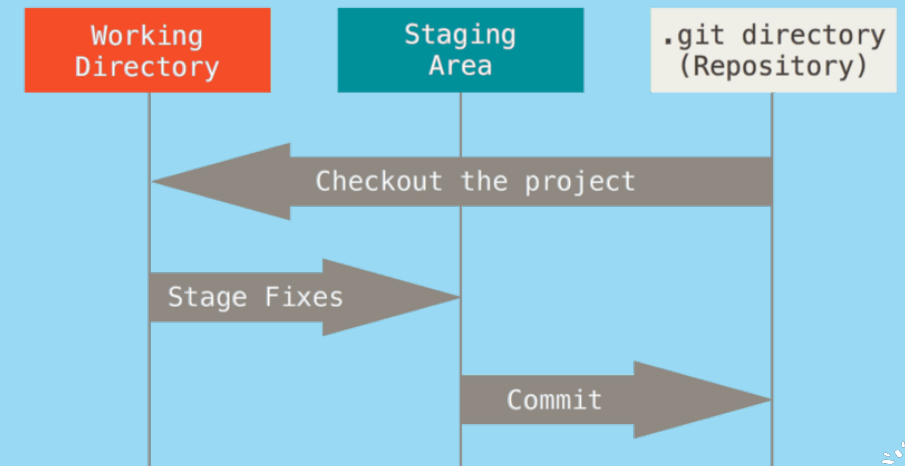


Version Control Systems (VCS) – Revision Graph



Git (2005)

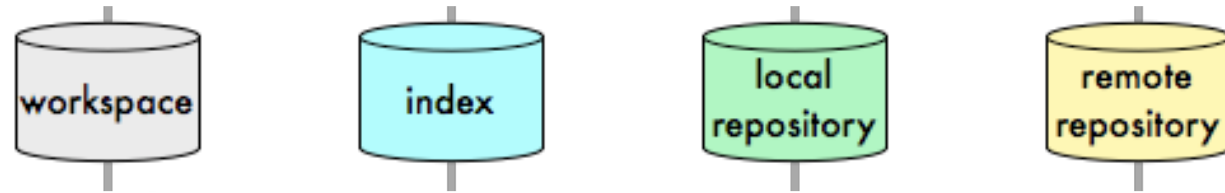
- Distributed version control system by Linus Torvalds
- Every git directory on any computer is a full-fledged repository with complete history and version tracking abilities
- Git considers three file states:
 - Committed
 - Modified
 - Staged



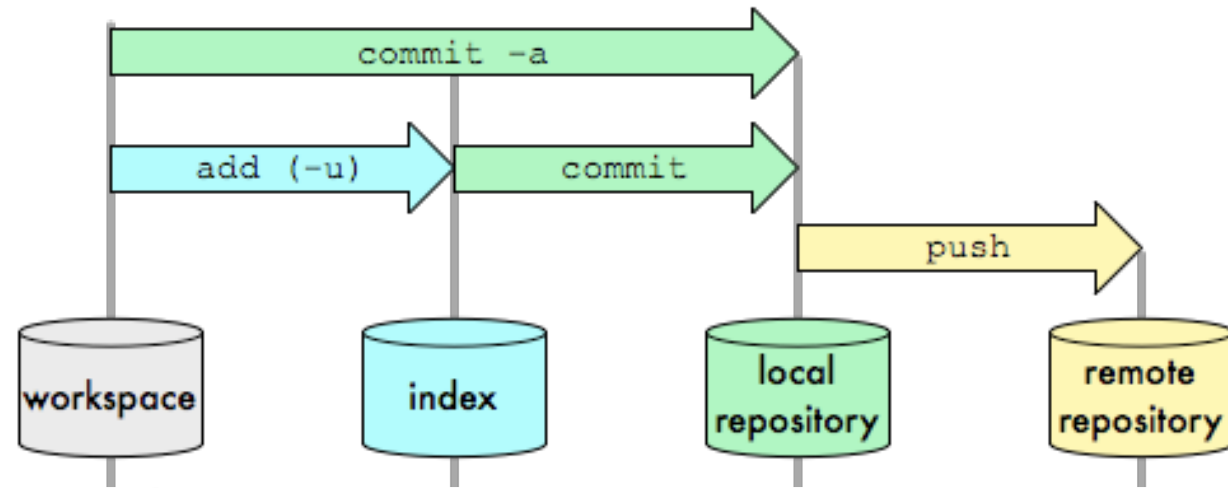
Git Remotes

- In practice: Two types of repositories:
 - Client-side repositories (Developer machines)
 - Server-side repositories (Github, Gitlab, ...)
- **Fetch (pull)** and **push** commands sync up two repositories
- When cloning from a repository, this repository is labeled as **origin**
- Most git workflows only fetch/push to/from origin
- As origin is usually a server-side repository, this is similar to a centralized version control system
- Multiple upstream repositories are possible
- Origin may also have an upstream repository itself

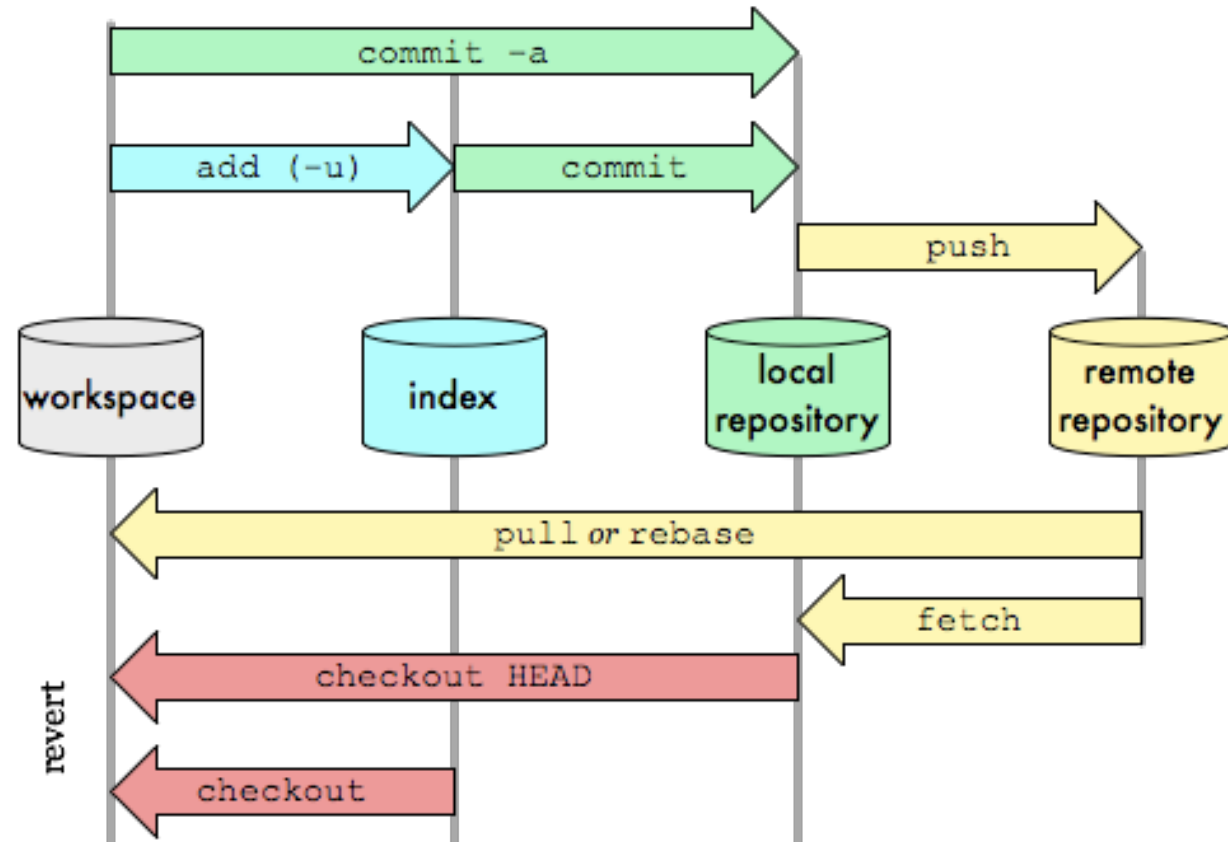
Git Repositories (1/3)



Git Repositories (2/3)



Git Repositories (3/3)



Git Example

```
$ mkdir test-project
$ cd test-project
$ git init
Initialized empty Git repository in .git/
$ echo 'hello world' > file.txt
$ git add .
$ git commit -m "initial commit"
[master (root-commit) 54196cc] initial commit
1 file changed, 1 insertion(+) create mode 100644 file.txt
$ echo 'hello world!' > file.txt
$ git commit -a -m "add emphasis"
[master c4d59f3] add emphasis
1 file changed, 1 insertion(+), 1 deletion(-)
$ git remote add origin github/my-repo
$ git push -u origin master
```

.gitignore

- Some files should not be committed to Git
 - Configuration files for your local setup (that may conflict with the setup of others)
 - Reproducible build artifacts, such as Jars, executables, generated code, ...
 - Libraries downloaded by your dependency management system
- Specify these files in .gitignore
 - Simple list of files
 - Supports wildcards (*)
 - Sample .gitignore files with common configurations for specific programming languages and frameworks are often generated by your IDE
 - Is committed to Git!

Demo

Collaboration Tools: Software Dependency Management

Quick Git Recap

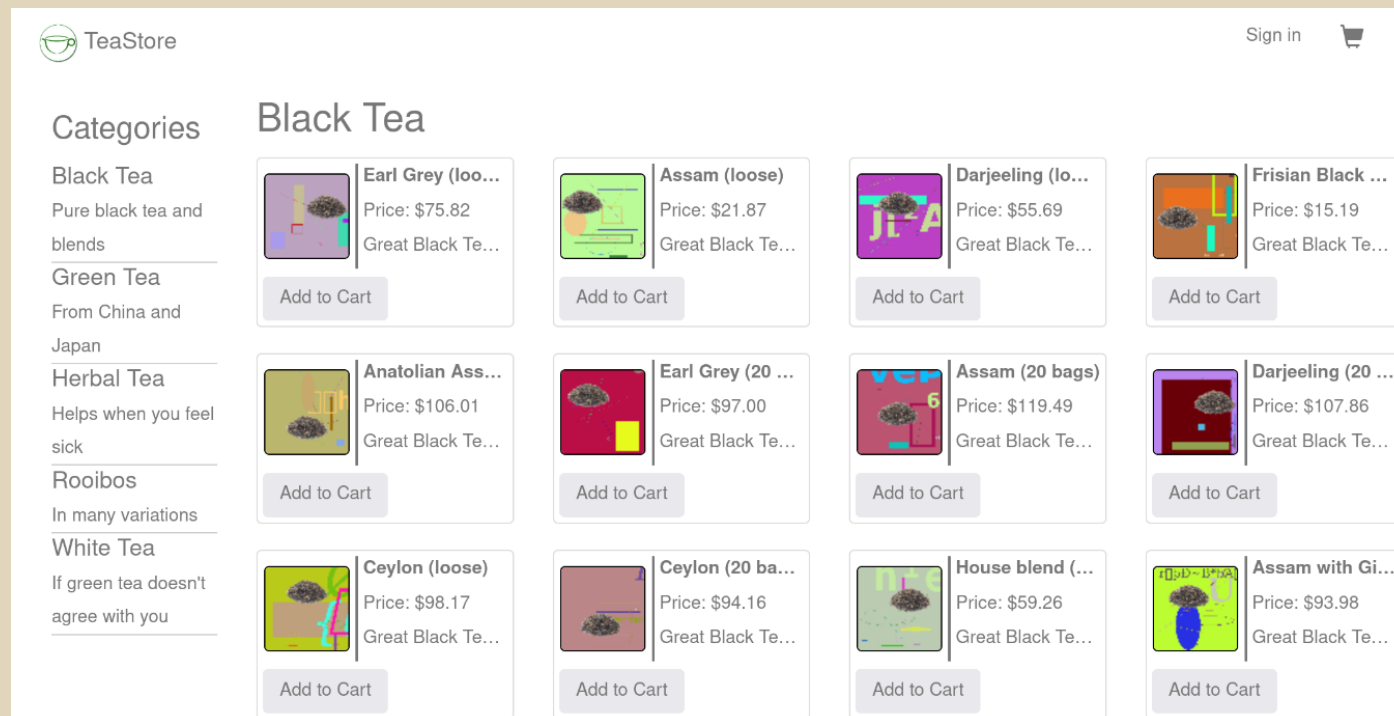


Dependency Management and Build Tools

Code Demo

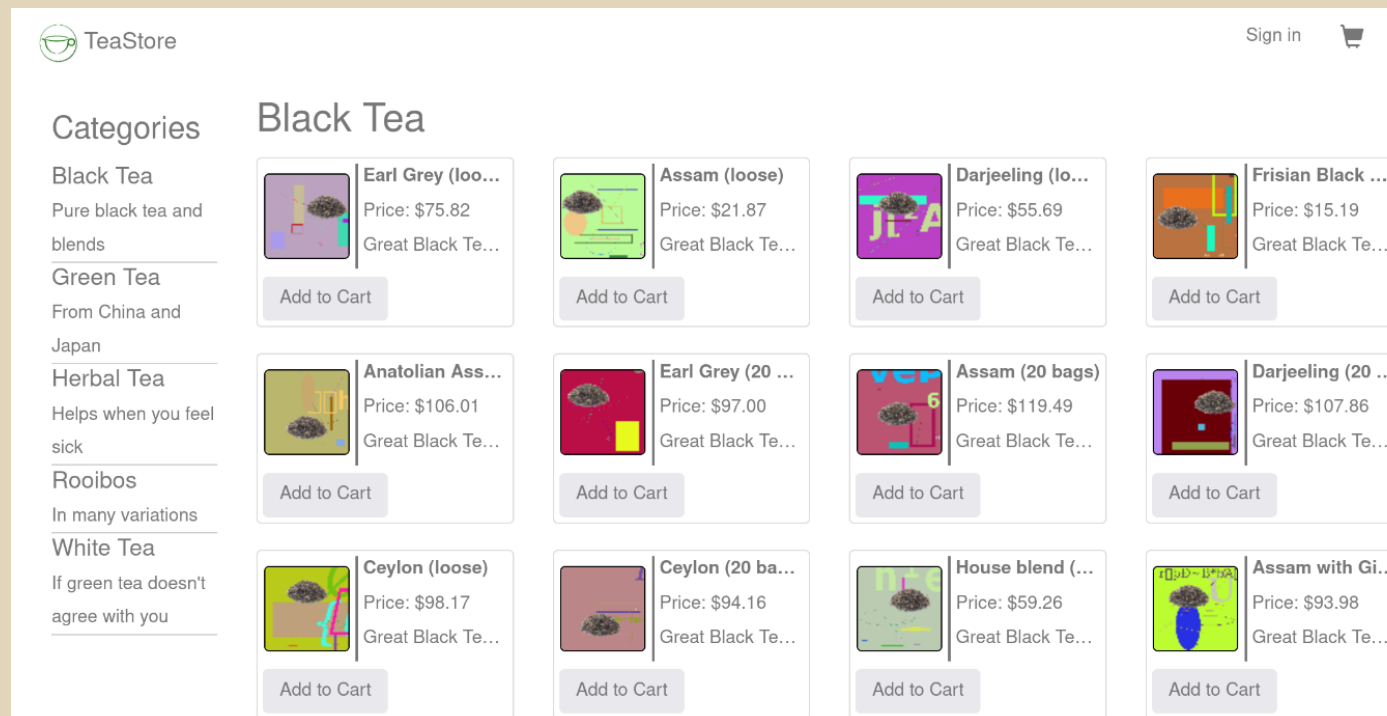
Question

What technical capabilities do we need to build a Tea Store?



Question

What technical capabilities (that we need) can we import as libraries?



The screenshot shows the TeaStore website interface. On the left, there is a 'Categories' sidebar with links for Black Tea, Green Tea, Herbal Tea, Rooibos, and White Tea. The main content area is titled 'Black Tea' and displays a grid of 12 tea products. Each product card includes a small image, the product name, the price, and an 'Add to Cart' button. The products are arranged in three rows and four columns.

Product Name	Price
Earl Grey (loose)	\$75.82
Assam (loose)	\$21.87
Darjeeling (loose)	\$55.69
Frisian Black ...	\$15.19
Anatolian Ass...	\$106.01
Earl Grey (20 ...)	\$97.00
Assam (20 bags)	\$119.49
Darjeeling (20 ...)	\$107.86
Ceylon (loose)	\$98.17
Ceylon (20 ba...)	\$94.16
House blend (...)	\$59.26
Assam with Gi...	\$93.98

Dependency Management – Problem

- Large Software Projects require **a lot** of libraries (software dependencies)
- Libraries have Versions
 - Some Versions have breaking changes
→ We need to manage versions
- Libraries need other libraries
 - E.g., WebServer-library requires networking library
- Some libraries are only needed during development, but not in production
 - E.g., JUnit
- We want to easily specify what libraries we are using (incl. version).
- We want our team members to use the same libraries we are using!

v1	v2
<code>MyLibraryClass.fancyCalc();</code>	<code>MyLibraryClass myObject = new MyLibraryClass(); myObject.facyCalc();</code>

Dependency Management Tools

- Software that manages ...
 - downloading dependencies/libraries
 - configuring correct dependency versions
 - resolving and downloading transitive dependencies (dependencies needed by other dependencies)
- Usually comes with one or more software repository, where dependencies are downloaded from
 - Commonly (but not always!) one central default repository
 - Alternative repositories / proxies can be configured
- Some dependency management tools also provide functionality for building, releasing, and deployment

Dependency Management Tools – Examples

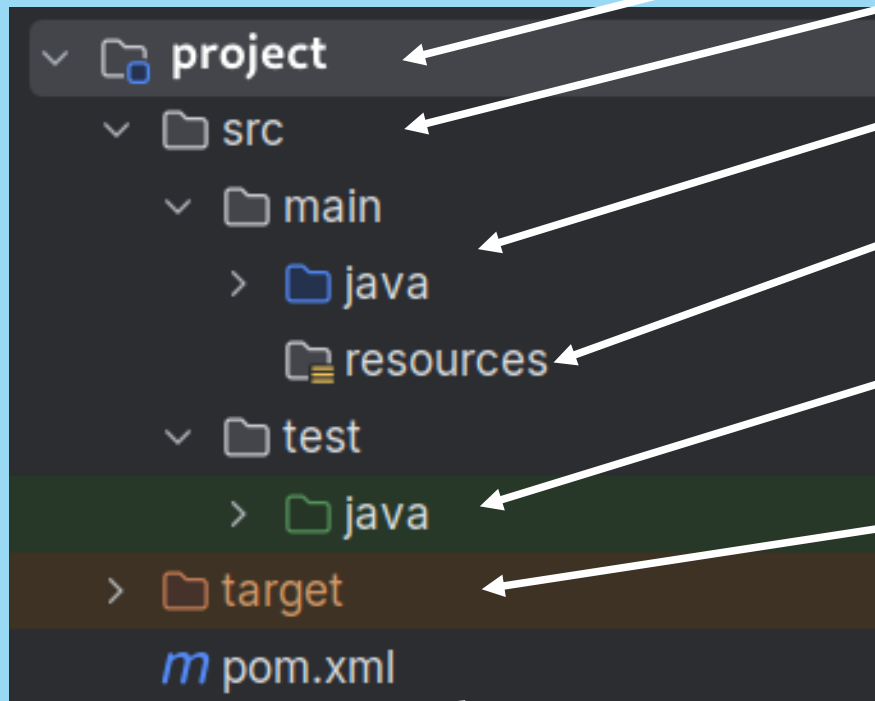
Java	Ant/Ivy, Maven, Gradle, SBT, ...
Go	go.mod
Rust	crate
Python	pip, conda
Javascript / Typescript	npm, yarn
Ruby	gem

Apache Maven

- Dependency management and build tool for Java
- Specifies dependencies in structured XML file
 - Project Object Model XML → **pom.xml**
 - Dependencies are specified using fixed and unique versions
 - Transitive dependencies are resolved using each dependency's pom.xml
- Central Repository: "Maven Central"
- Performs sequential steps according to the **Maven Lifecycle**
 - Each lifecycle step performed by a plugin
 - Additional plugins can be added and configured to run in the lifecycle



Maven – Project Structure



- Project root directory
- Source directory
- Java Sources
(to be packaged in the final application)
- Files to be packaged and shipped as part of the application (e.g. assets, internal configuration files)
- Test Sources
- *(missing)* Files to be packaged for tests only
- All output goes here!
 - Compiled .class-Files
 - Jars
 - Generated Code,
 - Etc.
- pom.xml

Maven – Declaring a Dependency

Example:

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-
jupiter</artifactId>
  <version>5.14.2</version>
  <scope>test</scope>
</dependency>
```

Gradle writes it like this:

org.mockito:mockito-junit-jupiter:5.14.12

- Dependency Scopes:
 - **compile** (Default)
 - available in all classpaths of a project
 - **provided**
 - available at compile time, but not packaged into runtime
 - **runtime**
 - not available at compile time, but packaged into runtime
 - **test**
 - only available for tests
 - **system**
 - JAR is on the local system; must be explicitly specified
 - **import**
 - pom.xml only, no JAR

Maven – Scope Examples

- **compile** (Default)
→ Web Framework, UI, Maths , ...
- **provided**
→ Application server libraries
- **runtime**
→ Database connection drivers
- **test**
→ JUnit and other testing frameworks/tools
- **system**
→ very, very rare
- **import**
→ Dependency management in frameworks (e.g., Spring); overrides versions for transient dependencies

```
<dependency>  
  <groupId>org.mockito</groupId>  
  <artifactId>mockito-junit-  
jupiter</artifactId>  
  <version>5.14.2</version>  
  <scope>test</scope>  
</dependency>
```

Maven – Remote and Local Repository

- All libraries can be downloaded from the remote repository
 - Usually Maven Central
- Downloaded to a local directory
 - Shared by all Maven projects
 - Default location: `~/.m2`
- Maven only downloads a dependency if it is not already in the local repository

Maven Lifecycle

- Ordered sequence of steps:

- *(Optional)* Clean
- Validate
- Compile
- Test
- Package
- Verify
- Install
- Deploy

- All Lifecycle steps at:
https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html#Lifecycle_Reference



Maven Lifecycle

- *(Optional)* **Clean**
Remove all generated output. i.e. delete the target directory.
- **Validate**
validate the project is correct and all necessary information is available; downloads dependencies, if needed
- **Compile**
compile the source code of the project
- **Test**
test the compiled source code using a suitable unit testing framework. These tests should not require the code be packaged or deployed

Maven Lifecycle

- **Package**
take the compiled code and package it in its distributable format, such as a JAR.
- **Verify**
run any checks on results of integration tests to ensure quality criteria are met
- **Install**
install the package into the local repository, for use as a dependency in other projects locally
- **Deploy**
copies the final package to the remote repository for sharing with other developers and projects

Maven – Command Line Interface (CLI)

- Can be run from the command line using
`mvn <lifecycle-step>`
- Runs all non-optional lifecycle steps up-to and including the given one
- Multiple lifecycle-steps can be provided
 - Mostly used to also specify otherwise optional steps to be executed (almost always: `clean`)

Maven – Some Useful Plugins

- Maven Surefire Plugin
 - Executes Unit Tests
 - Default Plugin for test step
- Maven JAR Plugin
 - Packages a JAR
 - Default Plugin for package step
- Maven Javadoc Plugin
 - Automatically generates Javadoc documentation
 - Can be configured to run right before compile (generate-sources step)
- Maven Shade Plugin
 - Packs library dependencies into “fat” JAR
 - Can be configured to run during package step

Which Maven lifecycle steps are executed by the following command?

`mvn clean test`

Dependency Management Systems – Fixed Versions vs. Version Ranges

- Maven and Gradle only support fixed and unique versions
- npm (Javascript), pip (Python), gem (Ruby), and others support version ranges

- Uses Semantic Versions (SemVer)

MAJOR.MINOR.PATCH
5.14.2

- Example: npm package.json

```
{
  "name": "npmProject",
  "version": "1.0.13",
  "dependencies": {
    "electron-is-dev": "^2.0.0",
    "electron-store": "^8.1.0"
  },
  "devDependencies": {
    "concurrently": "^8.2.2",
    "electron": "~32.0.1",
    "electron-builder": "^24.13.3",
  }
}
```

- ^ : Same MAJOR version

- ~ : Same MAJOR and MINOR version

Maven – The SNAPSHOT Version

Example:

```
<groupId>de.thab.cqt</groupId>  
<artifactId>my-project</artifactId>  
<version>1.0.0-SNAPSHOT</version>
```

- Artifacts with release (normal) versions are final
 - Maven assumes they will never change
 - Repositories will never allow re-upload of an already existing version
- SNAPSHOT indicates a development version
 - Content is expected to change until released
 - SNAPSHOT dependencies can be updated (by adding “-U” to the CLI command)

Advantages and disadvantages of version ranges?

Version Ranges: Lockfile

- Version ranges can lead to dependencies breaking
 - Mostly due to hidden breaking changes within dependencies in a given range
 - When working with others: “It works on my machine!”
- Solution: **Lockfile**
 - File generated by the dependency manager
 - Contains information on the exact version used
 - Using exact version specification
 - OR using cryptographic hash
 - Best practice: Commit lockfile to Git!

Testing when pushing to Git

Towards Continuous Integration

Testing on Git Push – Basic Idea

- Unit Tests should be run automatically on each Git Push
- On Push
 - Git Server sends Code and instructions to a build system
 - Code is compiled
 - Unit tests are run
 - (Optional) Executable is created and uploaded
- Angry automated Email when tests fail!
 - Or MS Teams message, or Slack message or Discord message ...

Question

Why is testing on a build server after every Git push a good idea?

Collaboration Tools: Software Dependency Management

Quick Git Recap

Dependency Management and Build Tools

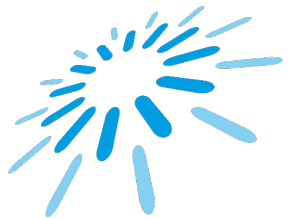


Code Demo



Until next time!

Collaboration, Quality, Test
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences